Done Is Not Correct: Measuring Silent Failures and Self-Verification Calibration When LLM Agents Take CAD Actions

Carson Rodrigues CARSON@CELABE.COM
Celabe

Clive Rodrigues RODRIGUESCLIVE8@GMAIL.COM
Hochschule Coburg, University of Applied Sciences

Aravind Reddy G ARAVIND99.G@GMAIL.COM
Independent

Reviewed on OpenReview: *under review*

Abstract

When a large language model (LLM) agent *builds* a mechanical part by writing parametric CAD code from a natural-language specification, syntactic success (the code runs, the geometry renders) decouples from semantic correctness (right dimensions, valid constraints, manufacturable features). We release a benchmark and harness that measure this gap directly: 13 specification tasks in three tiers whose every requirement maps to machine-checkable properties of the produced B-rep (dimensions within published tolerance bands, feature counts with axial extents, per-hole bolt-circle positions, mass-centre offsets, volume), measured exactly by the kernel; reference solutions and a calibration gate; and 519 graded agent attempts with final code, measured properties, verdicts under two oracle versions, full transcripts for every fixed-protocol tool-use attempt, and a 3D viewer. The exact, self-audited oracle isolates two quantities no existing CAD benchmark reports: the **silent-failure rate** (a valid, renderable solid, an explicit completion claim, and at least one failed semantic check) and **completion-claim calibration**. Across four models from four vendors (n=156 attempts per architecture over 13 tasks, cluster-robust intervals, paired tests), single-shot agents claimed completion on 98% of attempts and were silently wrong on 14.1% (task-cluster CI [5.8, 24.4]); a tool-use loop that enforces *measure-then-claim* cut silent failures to 3.2% (McNemar $p<10^{-4}$) and raised claim precision from 64% to 96%. A single-variable ablation on all four models attributes the change to the claim contract rather than to the feedback: with the legacy contract and identical feedback, silent failures were 14 of 156 against 5, and claim precision 68% against 96%. A planner–coder–checker pipeline matched that precision at lower coverage with no success gain. Verbalized confidence was poorly calibrated single-shot (ECE 0.32) and became informative under verification for three of four models. Context divergence between roles predicted episode breakdown (AUROC 0.86) but not valid-but-wrong parts. The benchmark audits itself: hardening our own oracle flipped 18 of 519 verdicts from pass to fail and none in reverse, exposing 17 of the corpus's 48 silent failures, while a blind expert audit found at most 3 of the 31 primary silent failures to be grader errors, so the rates are bounded on both sides. Scaling every tolerance band from a quarter to four times its published width changes no verdict in the verified conditions above half-width and leaves the single-shot to tool-use contrast intact (McNemar $p \leq 0.006$ at every scaling), so the conclusion does not rest on where the bands were drawn.

Keywords: agentic benchmarks, silent failure, LLM agents, computer-aided design, calibration, evaluation methodology, grader audit, text-to-CAD

1 Introduction

LLM agents increasingly act rather than advise: they write and execute code, call tools, and report back (Yao et al., 2022; Schick et al., 2023). Software engineering already has execution-grounded benchmarks for this regime (Jimenez et al., 2023), and a growing body of evidence that agent evaluation systematically overstates reliability: graders admit false positives (Yu et al., 2025; Zhu et al., 2025), headline accuracy hides inconsistency (Yao et al., 2024; Kapoor et al., 2024), web-agent progress shrinks under rigorous re-evaluation (Xue et al., 2025), and agents asked to estimate their own success are grossly overconfident (Kaddour et al., 2026).

Mechanical computer-aided design (CAD) is the next domain in this trajectory; it is also, we argue, the best instrumented one for studying agent reliability. A wave of recent work generates parametric CAD from text (Khan et al., 2024; Xie and Ju, 2025; Wang et al., 2026), increasingly with agentic feedback loops over a CAD kernel (Mallis et al., 2024; Yuan et al., 2026; Barkley et al., 2026; Ocker et al., 2025), and surveys now map the area (Zhang et al., 2025). Yet evaluation in this literature measures *generation quality*: Chamfer distance or IoU to a reference solid, execution success, or judge scores (Zhou et al., 2025; Alrashedy et al., 2024). None of it asks the reliability question that determines whether an engineer can delegate: *when the agent says the part is done, is it, and does the agent know?*

Crashes are cheap to catch: a spec that fails to execute announces itself. The expensive failure is a part that builds and renders plausibly while its bolt circle sits on the wrong pitch diameter, delivered with "done, confidence 0.97." Companion work by the first author (Rodrigues, 2026) shows that multi-agent LLM pipelines accumulate context drift, a divergence between agents' working beliefs that we also measure here (Section 3), and a growing body of work documents that humans over-rely on, and withdraw oversight from, action-taking AI systems (Schemmer et al., 2023); the self-correction literature explains why the agent's own check is not to be trusted without external feedback (Huang et al., 2023; Kamoi et al., 2024; Gou et al., 2023). CAD supplies what those settings lacked: exact measurement. The produced B-rep can be measured (dimensions, feature counts and extents, hole positions, mass properties, volume), so agent-side overclaiming can be separated from grader inadequacy to a degree execution-graded software benchmarks cannot (Yu et al., 2025); we audit and quantify the residual grader risk rather than assume it to be zero.

Contributions. (1) A reproducible benchmark and harness: 13 natural-language mechanical specification tasks in three tiers, each requirement mapped to machine-checkable properties of the produced B-rep with published tolerance bands, with reference solutions and a calibration gate, built on an open kernel (build123d/OpenCascade). It is released with every graded attempt (final code, measured properties, verdicts under both oracle versions, transcripts for the fixed-protocol runs), the grader, the harness, and a 3D viewer (Section 7). (2) A **claim-conditioned, self-auditing exact-oracle silent-failure rate coupled to completion-claim calibration**. The silent-failure rate is the conjunction of (i) valid renderable geometry, (ii) an explicit agent completion claim, and (iii) failure of at least one semantic check, graded by an exact kernel oracle that we audit against ourselves; it

is coupled to completion-claim calibration measured as ECE, reliability diagrams, AUROC, and Brier score (Guo et al., 2017; Tian et al., 2023; Kaddour et al., 2026). Conditioning on the agent's own claim and on a self-audited exact oracle is what separates this from the generation- and repair-quality scores of CADSmith (Barkley et al., 2026), the finite-element requirement-pass rate of Son et al. (2026), and the VLM-judged design-intent score of MUSE (Dong et al., 2026); we do not claim to be first to measure geometric correctness. (3) A controlled comparison of agent architectures (single-shot, tool-use with kernel measurement feedback, and a planner–coder–checker pipeline with context-drift instrumentation), with a single-variable ablation of the claim contract, testing whether feedback closes the syntactic–semantic gap or merely hides it (Cemri et al., 2025; Xie et al., 2026). (4) Findings with cluster-robust statistics over four model vendors: unverified completion claims carry a 14.1% silent-failure rate; enforcing measure-then-claim cuts it to 3.2%, and the ablation attributes the cut to the claim contract; an independent checker matches that claim precision at lower coverage; stated confidence becomes informative under verification for three of four models; context divergence predicts breakdown but not subtle wrongness; hardening our own oracle flipped 18 verdicts and none in reverse, so unmodelled properties can only raise the rates, while the expert audit bounds grader error at 3 of 31 silent failures; and the measurand is stable under scaling of every tolerance band (F7). We also report which of our pre-registered predictions the data did not support (Section 8).

2 Related work

Text-to-CAD generation and benchmarks. Datasets of parametric construction sequences (Wu et al., 2021; Willis et al., 2021; Koch et al., 2019; Seff et al., 2020; Kim et al., 2020) enabled one-shot text-to-CAD models (Khan et al., 2024; Xie and Ju, 2025; Guan et al., 2025; Xu et al., 2024) and benchmarks scoring generation quality across complexity tiers (Wang et al., 2026). Graders exist as training rewards (Zhou et al., 2025) and VLM-based self-checks (Alrashedy et al., 2024; Badagabettu et al., 2024). Agentic systems add kernel or visual feedback: tool-augmented VLLMs over FreeCAD (Mallis et al., 2024), clarification agents that resolve spec ambiguity (Yuan et al., 2026), multi-agent design teams (Ocker et al., 2025), agentic data synthesis at scale (Ataei et al., 2026), and workflow comparisons graded by human visual inspection (Schüpbach et al., 2025). CADSmith (Barkley et al., 2026) is closest to our infrastructure: it validates agent-generated CadQuery against exact kernel measurements and explicitly motivates itself by geometry that "silently diverged" from prompt intent, but treats silent divergence as a defect to engineer away. We treat it as the *measurand*: our benchmark quantifies how often agents ship it while claiming success, per architecture and per model. Two concurrent 2026 systems push geometric grading further along different axes. The FEA-feedback agent of Son et al. (2026) assembles multi-part STEP files from engineering briefs and validates them against typed physical and geometric requirements with a finite-element solver; this scores a requirement-pass rate rather than a claim-conditioned silent-failure rate, depends on a meshing and solver stack rather than a self-auditing exact-geometry oracle, and has no calibration axis. MUSE (Dong et al., 2026) grades manufacturability and assemblability of B-rep assemblies in three stages, but its design-intent stage is a VLM judge (a leaky oracle by the same argument we apply to visual self-checks), it scores single-shot generation, and it reports no silent-failure

rate or completion-claim calibration. 3DCodeBench (Gao et al., 2026) benchmarks agents that write procedural 3D code, but in the art-and-object domain with image- and shape-similarity plus judge scoring, not mechanical tolerances or a silent-failure rate. Against all of these, our contribution is a *claim-conditioned, self-auditing exact-oracle silent-failure rate coupled to completion-claim calibration*: we condition the metric on the agent's own completion claim, grade with an exact kernel oracle we audit against ourselves (Section 5, F6), and report the calibration of that claim, none of which these systems provide. We do not claim to be first to measure geometric correctness; we claim a distinct measurand, the first under an exact, self-audited (rather than judge-mediated) oracle. Topological property checks (Preintner et al., 2025) complement our dimensional and positional ones.

Agent reliability and overclaiming. Execution-grounded agent benchmarks measure end-state success (Jimenez et al., 2023; Yao et al., 2024), but their oracles leak: insufficient tests mislabel hundreds of SWE-bench patches as resolved (Yu et al., 2025), and systematic audits find task- and outcome-validity flaws across agentic benchmarks (Zhu et al., 2025; Kapoor et al., 2024; Xue et al., 2025). Orchestration-level work names silent failures (wrong-but-plausible outputs with no error signal) and engineers verifier layers against them (Babu and Agrawal, 2026); stress-testing frameworks measure consistency under perturbation (Gupta, 2026). Multi-agent failure taxonomies identify verification failure and false consensus as recurring modes (Cemri et al., 2025; Xie et al., 2026). Closest in construct, Advani (2026) characterize the same false-success phenomenon directly, measuring it at 45–48% on τ^2-bench and 75.8% on AppWorld and finding that LLM judges detect it only up to AUROC 0.65. CAD is the one agent-action domain where the correctness oracle is exact and machine-checkable rather than judge-mediated, so it is the only setting in which a silent-failure rate and completion-claim calibration can be measured without inheriting that judge-leakage ceiling. Our silent-failure rate is conditioned on the *agent's own completion claim* under an exact, self-audited geometric oracle, so it isolates agent-side overclaiming rather than grader weakness or injected faults.

Self-verification and calibration. LLMs can verbalize confidence (Lin et al., 2022), and verbalized confidence beats token probabilities for RLHF models (Tian et al., 2023), but elicitation studies find pervasive overconfidence (Xiong et al., 2023; Kadavath et al., 2022). Intrinsic self-correction does not work (Huang et al., 2023); external, tool-grounded feedback does (Gou et al., 2023; Kamoi et al., 2024). For code, confidence is poorly calibrated against execution-based correctness (Spiess et al., 2025). Closest to us, Kaddour et al. (2026) measure agents' predicted-versus-actual success on software tasks and find up to 55-point gaps, with post-execution self-review *worse* calibrated than pre-execution estimates. We instantiate this question in a domain with an exact geometric oracle, add the architecture axis, and couple calibration to a silent-failure rate; concurrent work measures silent failure with LLM judges that cap detection at AUROC 0.65 (Advani, 2026), whereas the exact kernel oracle here removes that ceiling.

3 The benchmark and harness

Task schema. A task is data, not code: `{id, tier, nl_spec, units, checks[]}`. Each check names a *kind* the grader knows how to evaluate against a measured-properties

dictionary: scalar comparisons with tolerance (`approx`, `range`, `equal`) over a fixed property vocabulary (bounding-box extents, volume, centre of mass, solid/face/edge counts), boolean validity checks (`valid_solid`, `watertight`), and specialized geometric checks (`cylinder_count`, `bolt_circle` with pitch-circle diameter and positional tolerance). Specs are written so that *every* requirement in the natural language maps to at least one computable check; this discipline is what makes the suite auto-gradable and auditable.

Tiers and variation. The v1 suite has 13 tasks in three tiers: **T1** single feature (flange with bolt circle, washer, bushing, four-hole plate, shaft collar), **T2** multi-feature (stepped shaft, flanged bushing, counterbored plate, three-step pin, pulley blank, L-bracket with non-coaxial holes), and **T4** edit-under-constraint (modify provided code: re-pattern a bolt circle; enlarge a bore under a material-volume cap). The suite has no multi-part assembly tier: that requires multi-solid measurement and is not part of this release. Every requirement is graded against a tolerance band; bands are engineering-informed, published per check with the suite, and their effect on the measurand is tested directly (F7). To our knowledge no existing CAD benchmark grades against tolerance bands rather than exact values or mesh similarity. The suite is small by design: 13 tasks and 122 checks are enough to separate agent architectures with paired tests across four vendors, and the released schema is what a larger suite extends (Section 8, roadmap).

Execution and measurement. Agents emit Python for the open `build123d` kernel (OpenCascade). Code runs in an isolated subprocess that builds the solid and measures kernel-truth properties; a crash or hang cannot take down the runner, and the only OpenCascade dependency is quarantined in that one module. Cylindrical features are detected by grouping cylindrical faces by their infinite supporting cylinder (radius + axis line) and merging groups by *axial extent*: overlapping spans are one physical feature (a hole whose face a boolean split at the seam), disjoint spans are separate features (coaxial bearing seats). The extractor is lenient on wrapper conventions (builder objects, lists of solids) and strict on geometry.

Agent conditions. **Single-shot**: one completion from the spec. **Tool-use**: a ReAct-style loop (Yao et al., 2022) in which the harness executes each code revision and returns the measured geometry of the solid: validity, bounding-box extents, volume, centre of mass, solid count, and, for every detected cylindrical feature, its radius, axis, centre, and axial extent. The feedback never includes the specification's targets, tolerance bands, or any pass/fail verdict: the agent sees what a kernel reports, not what the grader concludes, and must still map the specification onto those numbers itself. A completion claim that accompanies fresh, unverified code does not end the episode; the agent must see kernel measurements for its final code before its DONE is accepted, ensuring the condition actually delivers the feedback it is named for. **Multi-agent**: planner→coder→checker, where the checker judges from the identical kernel-measured property set (plus a watertightness flag) and never from the checks themselves, so the two verified conditions are information-matched and differ only in architecture. Each role ends its turn with a flat JSON of the governing dimensions it believes (`DIMS`); the per-round **Context Divergence Score (CDS)** is the mean pairwise disagreement over the union of stated dimension keys (a key counts as agreeing when present for both roles and within 2% relative tolerance; the planner's key vocabulary is propagated so naming variation is not scored as drift). CDS = 0 means perfectly aligned beliefs, 1

means none shared; it operationalizes context divergence for this domain and tests whether verification-stage failures and false consensus (Cemri et al., 2025; Xie et al., 2026) predict silent failures. In every condition the agent is instructed to report DONE plus a verbalized confidence in $[0, 1]$ when it believes all requirements are met; verbalized elicitation follows Tian et al. (2023).

Grader soundness. Every task ships a reference solution (never shown to agents). A calibration gate requires each reference solution to pass every one of its task's checks before any agent run is scored; a stratified sample of 81 valid-solid agent outputs (all 31 silent failures in the regraded corpus, 25 withheld-but-wrong parts, and 25 randomly drawn grader-passes) was packaged with rendered geometry and blind-coded by an independent practising mechanical (CAD) engineer and co-author against the released specifications, scoring each part pass/fail without sight of the grader verdict. Expert-versus-grader agreement is *substantial*: Cohen's $\kappa = 0.64$ (raw agreement 82.7%, $n = 81$). Ten of the fourteen disagreements trace to a single task, T2-LBRACKET, whose natural-language specification admits two readings of the base depth (the 8 mm wall inside the stated 60×40 footprint, giving an overall depth of 40 mm, versus a full 60×40 base with the wall added outboard, giving 48 mm). Standard angle-bracket practice dimensions legs heel-to-toe, i.e. as overall lengths, and our own wording fixes the vertical leg as an "overall height," so the 40 mm base depth is likewise overall; the grader adopted this reading and the expert the other. This is a finding about natural-language underspecification rather than a grader defect: excluding that one ambiguous task, agreement rises to $\kappa = 0.87$ (raw 93.5%, $n = 62$). The full agreement table, per-item disagreements, and a reproducible computation from the released sheet are in the supplement. We designed to the Agentic Benchmark Checklist (Zhu et al., 2025): outcome validity comes from kernel-measured properties rather than tests or judges, and task validity from the requirement-to-check mapping plus expert review.

4 Metrics

Let an attempt produce properties p, grade $\text{pass}(p) \in \{0, 1\}$ (all checks within tolerance), validity $v(p) \in \{0, 1\}$ (renderable, topologically valid solid), claim $d \in \{0, 1\}$ (agent reported DONE), and confidence $c \in [0, 1]$.

- **Success rate** Pr[pass] and **success@**k (any of k seeds passes), plus **pass^**k consistency (all of k seeds pass) (Yao et al., 2024); capability and reliability are reported separately.

- **Silent-failure rate** $= \Pr[\, v \wedge d \wedge \neg\text{pass}\,]$: a valid, renderable solid, an explicit completion claim, and at least one failed semantic check. Loud failures ($\neg v$) and honest non-claims ($\neg d$) are excluded by construction, so the metric isolates agent-side overclaiming under a sound oracle.

- **Completion-claim calibration**: ECE with equal-mass bins and reliability diagrams (Guo et al., 2017) over (c, pass) pairs; **AUROC** of c as a discriminator of pass/fail (informative at low base rates (Kaddour et al., 2026)); Brier score.

- **Claim rate** Pr[d] and **claim precision** Pr[pass | d]: the architecture-neutral pair a user experiences, reported alongside the silent-failure rate (interference and manufacturability checks arrive with the assembly tier in v2 and are not reported here).
- **Cost**: input and output tokens and tool calls per attempt.

5 Results

Populations. A mid-study audit (Section 8) found a protocol loophole in the original tool-use loop and permissive checks in the original oracle; we hardened both, re-executed and regraded every stored attempt, and re-ran tool-use under the fixed protocol where budget allowed. We therefore report three populations honestly: **primary**: claude-haiku-4-5, claude-sonnet-4-6, deepseek-v3.2, and qwen3.7-max (four vendors), complete 13-task coverage, hardened oracle, fixed-protocol tool-use (n=156 per architecture; all headline claims and statistics come from here); **ablation**: a tool-use arm rerun under the legacy claim contract with identical feedback on all four models (n=156; the sonnet arm was run through the OpenRouter gateway to the same model). An incomplete exploratory run of a fifth model (gpt-5.5, an easy-tier prefix under the original protocol) is included in the release for completeness but is not used in this paper. Every rate below carries a Wilson 95% interval; bracketed intervals on contrasts are task-cluster bootstraps; paired tests are exact McNemar over (task, model, repetition) triples; a GEE logistic model with task clusters confirms each headline contrast (with only 13 clusters its sandwich standard errors are anti-conservative, so the cluster bootstrap is the primary interval and the GEE is corroborating). The paper tests two pre-specified contrasts (single-shot versus each verified condition); every other comparison is reported descriptively, without a multiplicity correction, and should be read that way. The three repetitions are resamples at provider-default decoding (no seed control); attempts within a task are correlated, and the cluster intervals reflect that. The headline contrast holds within every repetition taken alone: single-shot silent failures were 5, 9 and 8 of 52 per repetition against 1, 4 and 0 for enforced tool-use and 1, 2 and 1 for the checker pipeline, so no single resample drives it.

F1: unverified claims are the hazard; enforced verification almost removes it. Single-shot agents claimed DONE on 153/156 attempts; 22 (14.1%; task-cluster CI [5.8, 24.4], Wilson [9.5, 20.4]) were silent failures, and claim precision (Pr[pass | claim], the architecture-neutral quantity a user experiences) was 64.0%. The measure-then-claim tool-use loop cut silent failures to 5/156 (3.2%) against single-shot's 22/156 (McNemar over (task, model, repetition) triples, removed 18, created 1, $p<10^{-4}$; the margins, not the pairing, carry the meaning given uncontrolled decoding; silent-rate difference CI [3.9, 19.2] pp), with claim precision 96.0% and a selective claim rate of 80.8%. Success also rose (64.1%→77.6%; paired McNemar fixed 25, broken 4, $p<10^{-4}$; GEE task-clustered p=0.00014). The single-variable ablation isolates the contract: rerunning tool-use under the legacy claim contract with *identical feedback* on all four models (n=156) produced 14 silent failures against the fixed contract's 5 (Fisher p=0.056; haiku 6 vs 1, sonnet 4 vs 0, deepseek 4 vs 2, qwen 0 vs 2), claim precision of 67.6% against 96.0% ($p<10^{-9}$), and success of 64.1% against 77.6% (p=0.013). The silent-failure difference on its own is marginal and one model runs the other way on small counts; the contract's effect on what gets claimed is not marginal. It converges

Model	Condition	n	Success	Silent	Conf.	ECE	Steps
claude-haiku-4-5	single-shot	39	46%	21%	0.93	0.498	1.0
claude-haiku-4-5	tool-use	39	62%	3%	0.93	0.102	3.0
claude-haiku-4-5	multi-agent	39	41%	0%	0.48	0.061	2.6
claude-sonnet-4-6	single-shot	39	90%	10%	0.95	0.084	1.0
claude-sonnet-4-6	tool-use	39	100%	0%	0.98	0.016	2.2
claude-sonnet-4-6	multi-agent	39	95%	3%	0.96	0.030	1.1
deepseek-v3.2	single-shot	39	36%	18%	0.99	0.671	1.0
deepseek-v3.2	tool-use	39	54%	5%	0.98	0.279	3.3
deepseek-v3.2	multi-agent	39	38%	0%	0.56	0.195	2.8
qwen3.7-max	single-shot	39	85%	8%	1.00	0.149	1.0
qwen3.7-max	tool-use	39	95%	5%	1.00	0.051	2.3
qwen3.7-max	multi-agent	39	72%	8%	0.97	0.252	1.7

Table 1: Primary population. *Silent* = valid renderable solid + explicit completion claim + ≥1 failed semantic check. ECE = equal-mass 5-bin expected calibration error. Counts, not just rates: silent failures are 22/156, 5/156, 4/156 by condition.

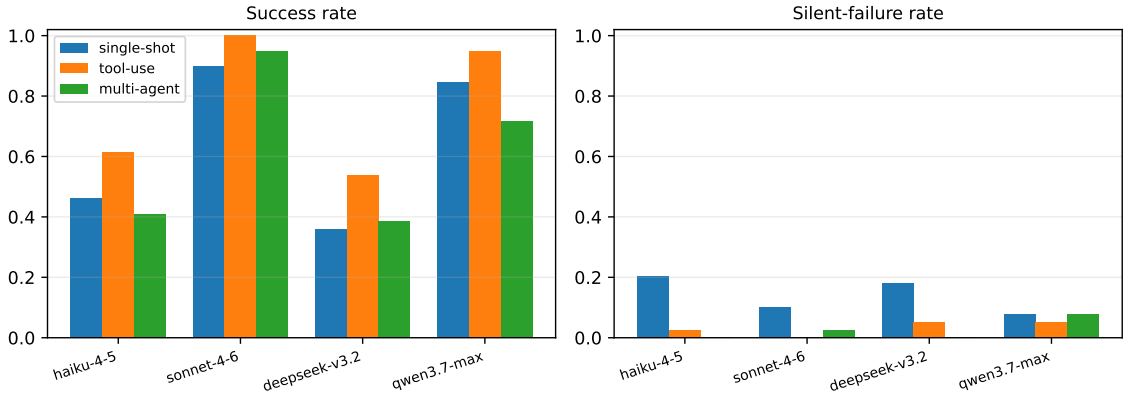

Figure 1: Success and silent-failure rates by model and agent condition (primary population).

with the run-level sensitivity: the original loophole runs put most tool-use silent failures at unverified claim points, and two harness variables changed between those runs, which is why the ablation cell was run at all. This sharpens, and partially retracts, the reading of our own pilot and the optimistic framing of feedback-loop systems (Barkley et al., 2026): what closes the semantic gap is not iteration but verification discipline at claim time.

F2: the independent checker trades coverage for precision without improving capability. The planner–coder–checker pipeline also reached 95.5% claim precision with 4/156 silent failures (McNemar vs. single-shot: 19 removed, 1 created, $p<10^{-4}$), but at a

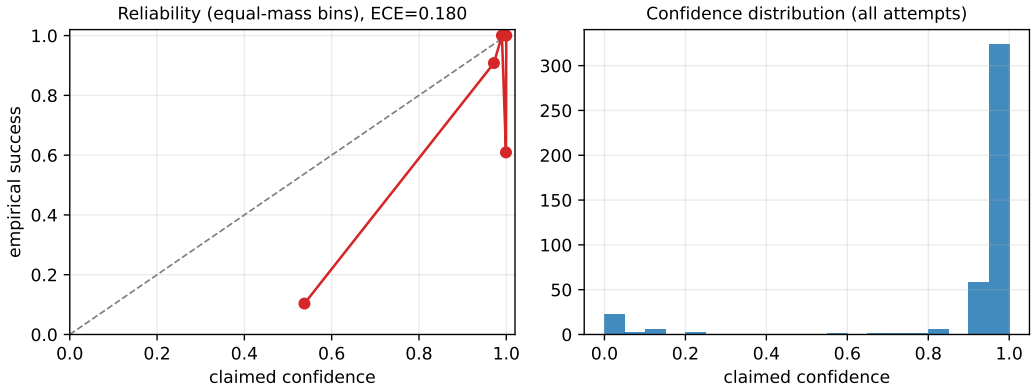

Figure 2: Reliability diagram and confidence distribution, primary population.

57.1% claim rate and with no success gain over single-shot (61.5% vs. 64.1%). Decomposing its 60 failed episodes: 53 never produced a valid solid (the coder, fed second-hand checker feedback rather than raw kernel traces, crashes more), 3 were valid-but-wrong parts correctly held back, 4 were silent failures; the checker also withheld claims on 11 episodes whose parts were actually correct. In selective-prediction terms (Geifman and El-Yaniv, 2017; Mozannar and Sontag, 2020), enforced tool-use and the checker pipeline sit at different points on the same risk–coverage curve (81% coverage at 4.0% conditional risk vs. 57% at 4.5%); on this suite the architecturally simpler tool-use point dominates. The checker's verdict is also a different claim-bearer than the generator's own DONE; claim precision is the cross-architecture metric precisely because it is agnostic to who claims.

F3: with verification discipline, stated confidence becomes informative. Single-shot confidence is saturated (claim rate 98%, mean 0.94) and poorly calibrated (ECE 0.324); its silent failures carried mean confidence 0.93 (min 0.60, pooled over all conditions). Under enforced tool-use and the checker pipeline, ECE falls to 0.091 and 0.110, and per-model AUROC (the pooling-robust quantity) rises for three of the four models (haiku 0.72→0.98/0.99; sonnet 0.93→1.00 on few negatives; deepseek 0.65→0.83/0.93). The fourth, qwen3.7-max, stays at chance (0.50–0.55): its confidence is saturated at 1.0 regardless of outcome, so verification discipline improved its claims but not its stated uncertainty; the two are separable, which is itself a finding. The checker's confidence restricted to valid-solid episodes discriminates at AUROC 0.68 (7 negatives). We do not claim the architectures *teach* calibration; the verified conditions let the agent condition its claim on measurements, which is exactly the point.

F4: context divergence tracks episode breakdown but adds nothing over episode length, and misses subtle wrongness. Episode-mean CDS (defined in Section 3) separates passing from failing multi-agent episodes (0.105 vs. 0.518 for never-built solids, Mann–Whitney $p<10^{-15}$; AUROC 0.86 as an episode-failure predictor). That number needs its baselines: the number of coder–checker rounds predicts the same outcome at AUROC 0.87, and the validity flag alone at 0.94, so CDS adds no predictive value over episode length. Its interest is diagnostic rather than predictive: it names which dimension the roles disagree

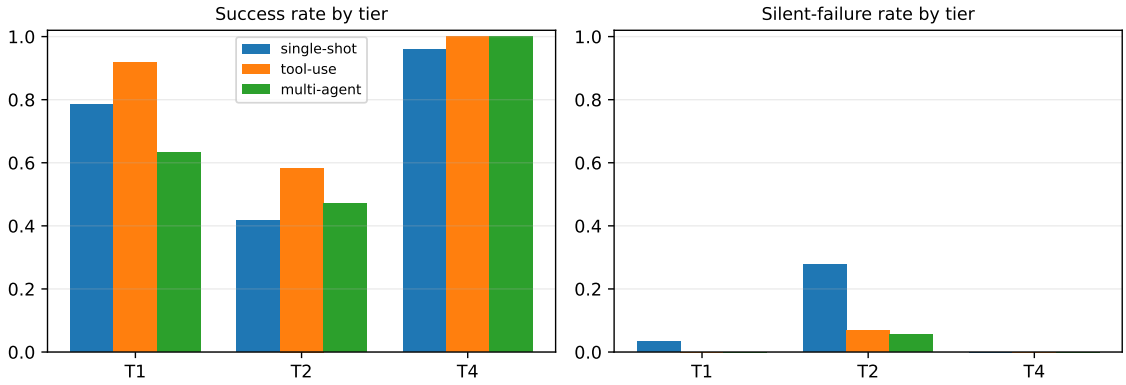

Figure 3: Success and silent-failure rates by tier and condition (primary population).

about while the episode is still running. Stratifying by validity removes even that: the seven valid-but-wrong episodes average CDS 0.107, indistinguishable from passes (0.105). Drift between the roles' stated dimensions is an early breakdown signal, but it does not flag the plausible-wrong parts that motivate this paper. We report this null explicitly.

F5: silent failures live in internal multi-feature geometry. All 31 primary silent failures are T1/T2 parts; 19 of 31 come from two tasks (the L-bracket and the counterbored plate: internal features and non-coaxial holes), and the two T4 edit tasks produced none (Figure 3; exploratory slices, not tested). Hazard tracks internal-feature complexity rather than headline task difficulty. Because the L-bracket specification admits two readings of the base depth (Section 3), we also report the primary contrast without that task: on the remaining 12 tasks (n=144 per condition) single-shot produced 16 silent failures (11.1%), enforced tool-use 2 (1.4%), and the checker pipeline none, with the paired single-shot versus tool-use test unchanged in direction (15 removed, 1 created, McNemar p=0.0005). The ambiguous task inflates the absolute rates and narrows, rather than creates, the gap between conditions.

F6: the oracle audits itself, and rates are bounded on both sides. Prompted by an adversarial engineering review of our own suite, we hardened the oracle: axial extents on every cylindrical check (blind holes, transposed sections), per-hole radial deviation plus pattern-centre and angular-uniformity constraints on bolt circles, translation-invariant mass-centre offsets (mirrored and wrong-face features), body counts, and ±8% volume bands. Re-executing and regrading all 519 stored attempts flipped 18 verdicts pass→fail and **zero** fail→pass, exposing 17 silent failures the original checks had hidden, 35% of the 48 silent failures in the regraded corpus. Property oracles are necessary-condition screens: any property they do not model can only add failures, so along that axis the measured rates are lower bounds. Grader false negatives push the other way, and the blind expert audit (Section 3) bounds them: outside the spec-ambiguous task the expert overturned 4 of 62 verdicts, all from grader-fail to expert-pass, and 3 of those 4 had been counted as silent failures (two single-shot counterbored plates, one tool-use stepped shaft). Reading the expert as ground truth for those parts moves the primary rates to 20/156 (12.8%) single-shot and 4/156

Band scale	SS silent	TU silent	MA silent	SS success	TU success	SS−TU (pp), McNemar p
×0.25 (tight)	33/156 (21.1%)	17/156 (10.9%)	14/156 (9.0%)	57.0%	69.9%	10.3, <0.001
×0.5	22/156 (14.1%)	5/156 (3.2%)	4/156 (2.6%)	64.1%	77.6%	10.9, <0.001
×1 (published)	22/156 (14.1%)	5/156 (3.2%)	4/156 (2.6%)	64.1%	77.6%	10.9, <0.001
×2	20/156 (12.8%)	5/156 (3.2%)	4/156 (2.6%)	65.4%	77.6%	9.6, <0.001
×4 (loose)	15/156 (9.6%)	5/156 (3.2%)	4/156 (2.6%)	68.6%	77.6%	6.4, 0.006

Table 2: Tolerance-band sensitivity, primary population (n=156 per condition). Every band is scaled by the factor in the first column; SS single-shot, TU tool-use, MA multi-agent. The published bands reproduce the stored verdicts exactly.

(2.6%) tool-use; no grader pass was overturned. The rates are therefore bounded on both sides, and benchmark authors should expect their own graders to fail the audit they apply to agents.

F7: the measurand is stable under tolerance-band scaling. Every verdict depends on the published bands, so we regraded the primary population with every band (dimensional tolerances, cylinder radius and axial-extent tolerances, bolt-circle radial, centre and angular tolerances, and the half-width of every range check) scaled by a common factor (Table 2). At the published bands the offline regrade reproduces all 468 stored verdicts. Halving every band changes no verdict: every part that passes at the published bands also passes at half of them, so the silent failures are gross errors (a missing counterbore, a hole on the wrong pitch circle, a wall added outboard), not marginal ones. Doubling the bands forgives two of the 22 single-shot silent failures and quadrupling forgives seven; no tool-use or checker verdict moves at any scaling above half-width, and the single-shot minus tool-use contrast survives every scaling from ×0.25 to ×4 (10.3 to 6.4 pp, McNemar $p \leq 0.006$). Tightening to a quarter of the published bands raises silent failures in every condition (33, 17 and 14 of 156), because parts correct to the millimetre are not correct to a quarter-millimetre, and leaves the ordering of conditions unchanged. The conclusion does not rest on where the bands were drawn.

Protocol sensitivity. The retained old-protocol tool-use runs serve as sensitivity data: deepseek-v3.2 under the loophole contract showed 11/39 silent failures versus 2/39 under the fixed contract, the largest per-model swing and the pattern F1 predicts for a weak self-verifier. These rows are consistent with the primary findings but are not statistically interpreted.

Costs and consistency. Mean tokens per attempt (input/output): 211/735 single-shot, 2,821/1,549 tool-use, 2,279/2,787 multi-agent: verification discipline costs roughly an order of magnitude more input tokens than single-shot. Pooled success@3 is 81.4% vs. pass^3 53.8%: best-of-k reporting overstates dependable capability (Yao et al., 2024).

Worked examples. Among the five fixed-protocol tool-use silent failures, a representative one is a claude-haiku-4-5 L-bracket: valid, rendered, claimed at confidence 0.95, failing its depth, volume, and mass-centre checks. In single-shot, the same model produced a counterbored plate with the counterbore cut from the wrong face, caught only by the hardened mass-centre check; and deepseek-v3.2 (secondary) shipped a six-bolt flange failing

bore and bolt-circle checks at "DONE, confidence 1.0." Every attempt is inspectable in the released 3D viewer.

6 Error modes

Separating failure layers on the primary population: of 151 failed attempts, 95 never produced a valid solid (execution or modeling aborts), and 56 produced valid geometry that failed semantic checks. Among the latter only, the dominant signatures are missing or mis-sized internal features (counterbore depth/face, hole extents) and dimensional errors on multi-feature parts; envelope dimensions are rarely wrong. We do not over-interpret multi-label counts across layers: a crashed attempt fails every check vacuously, and earlier drafts of this paper made that conflation; the released analysis code now reports the layers separately.

A practitioner-coded failure taxonomy with inter-rater agreement, following Cemri et al. (2025), is not part of this paper; the released transcripts are the material for it (Section 8, roadmap).

7 Availability, licensing, and maintenance

Everything the paper reports is derived from the released artifact at `https://github.com/rodriguescarson/cad-silent-failure-bench`, archived with a versioned DOI at `https://doi.org/10.5281/zenodo.22286324`. It contains: the 13 task specifications (JSON) with their 122 checks and published tolerance bands; the 13 reference solutions; the grader, the measurement extractor, the executor, and the three agent harnesses; every graded attempt in the corpus (519 attempts in the full population, of which 468 form the primary population, plus the 78-attempt ablation arm and the 60-attempt pilot) as JSON records carrying the final code, the measured-properties dictionary, the verdict under the original and the hardened oracle, the agent's completion claim and stated confidence, token counts, and the per-round context-divergence scores for multi-agent episodes; full multi-turn transcripts for the 156 fixed-protocol tool-use attempts and the 156 ablation attempts; the expert scoring sheet behind the κ analysis; the analysis scripts that produce every number, table, and figure in this paper; and a static 3D viewer of every attempt against its reference. Code is released under the MIT licence; task specifications, reference solutions, attempt records, and transcripts under CC BY 4.0. The authors bear all responsibility for the released material; it contains no personal data, and the only human judgement it records is a co-author's blind scoring of geometry. Hosting is a public GitHub repository for the living suite and a Zenodo record for the frozen version reported here (suite v1.0); the record cited in this paper is immutable, later suite versions are tagged releases with their own DOIs, and corrections are logged in an errata file rather than applied silently. The first author maintains the repository and answers issues; the release will be kept runnable against the pinned kernel version for at least three years. A datasheet (Gebru et al., 2021) and a reproducibility checklist are in the appendix.

8 Discussion, limitations, and roadmap

What the results mean. In the primary population, silent-failure rates tracked one variable: whether the agent could say DONE without having seen measurements of its final geometry. Unverified claiming ran at an 18.5% conditional silent rate among valid claimed parts; both verified configurations sat near 2%. The secondary models are directionally consistent, with deepseek's old-protocol tool-use the worst case (11 silent failures in 34 claims). That reframes the design question for engineering copilots from "which agent pattern?" to "where does the claim sit relative to the measurement?", a question of harness contract, cheap to enforce and testable, echoing the external-feedback conclusions of the self-correction literature (Gou et al., 2023; Kamoi et al., 2024). The same lesson applied to us: our first harness contained the loophole and our first oracle the leniency that this paper warns about, and both materially moved the numbers. We kept the audit trail in the paper because a reliability benchmark should not exempt its own grader from the scrutiny it applies to agents.

Pre-registered predictions and what the data did to them. The study plan fixed three predictions before any agent was run, and the released plan file carries them verbatim. **P1**, that tool-use with kernel feedback beats single-shot on dimensional correctness, is supported (F1), with the ablation adding the attribution to the claim contract. **P2**, that context drift between roles raises the constraint-violation rate on multi-feature specs, is not supported: divergence predicts episodes that never build a solid, and the seven valid-but-wrong episodes are indistinguishable from passes on CDS (F4). **P3**, the plan's headline, predicted that the silent-failure rate improves far less with model capability than syntactic success does. The single-shot data do not support it as stated. Among valid claimed parts, the conditional silent rate falls from 31% (haiku, 8/26) and 37% (deepseek, 7/19) to 10% (sonnet, 4/39) and 8% (qwen, 3/36) as the valid-solid rate rises from 54–67% to 92–100%; both improve together. What survives is narrower and, we think, the more useful statement: the strongest single-shot model still ships one silently wrong part in ten claimed valid parts, and the reduction to 4–5% comes from the claim contract, not from capability (the fixed-protocol tool-use rate is 5/126 valid claims pooled, with sonnet at 0/39 and qwen at 2/39). Two pre-specified analysis choices also changed. The plan named a mixed-effects model with task and seed random effects; provider-default decoding gives no seed control, so the reported models are task-clustered GEE and cluster bootstraps, which estimate the same contrasts without a seed term. The plan set an expert-versus-grader agreement target of $\kappa \geq 0.8$; the full 81-part sample reached 0.64, and 0.87 only after excluding the one task whose specification the expert read differently (Section 3). We report the miss rather than the post-hoc subset alone.

Would a machinist catch these anyway? In a conventional drawing–CAM–inspection chain, many of these specific failures would surface downstream: a bolt circle off pattern fails first-article inspection; a blind hole is caught at assembly. Three answers. First, in model-based-definition and direct-to-CAM workflows the model *is* the spec; inspection then verifies against the wrong geometry, and the error self-certifies. Second, even when caught, the cost is scrap, rework, and lead time; the copilot's value proposition collapses if every output needs full manual re-inspection, which is precisely the over-delegation regime

(Schemmer et al., 2023). Third, the failures concentrate in internal multi-feature geometry (F5), the class that slips visual review. We concede the converse: envelope-dimension errors, which downstream processes catch reliably, are also the ones our agents rarely make.

Tolerance policy. The bands are *model-correctness* bands, not functional fit tolerances: a generated part 0.3 mm off its specified bushing OD is a wrong model, not a loose fit, and grading fit dimensions functionally (IT6–7, tens of microns) would test arithmetic nobody asked the agent to do. Bands are published per check with the suite, and F7 shows that scaling all of them by a factor of four in either direction does not change the paper's contrasts. A single declared policy (ISO 2768 class $\times$ stated multiplier) would remove the residual inconsistency a careful reader can find across tasks; it is on the roadmap, not in this release.

Contamination and headroom. The parts are canonical and abundantly represented in training corpora; memorization should help agents be *right* when they claim, so measured silent-failure rates on these parts are, if anything, optimistic for novel geometry, the direction that strengthens the warning but weakens absolute rates. Frontier models saturate the suite's success axis (sonnet at 90–100% across conditions), so the benchmark's discriminating axis is reliability, not capability; parametric re-instantiation (schema-supported, v2) is the designed contamination control.

Limitations. The suite is 13 tasks and 122 checks. That is enough to separate architectures with paired tests across four vendors, and not enough to rank models finely or to estimate absolute rates that would transfer to a different task mix; silent failures concentrate in two tasks (F5), so the mix matters. Results are a dated snapshot of pinned model versions. The primary population is four models from four vendors. Three repetitions per cell at provider-default decoding bound the variance estimates; 13 task clusters bound the cluster bootstrap. Thirteen of the 31 primary silent failures come from the one task whose specification admits two readings; the headline contrast survives excluding it (F5), and absolute rates on that task should be read as an upper bound on agent error. The oracle, though hardened and audited, remains a necessary-condition screen: silent-failure rates are bounded below by unmodelled properties and above by grader false negatives, which the expert audit puts at 3 of 31 (F6). Blind expert hand-scoring of a stratified 81-part sample bounds the residual false-positive and orientation-driven false-negative directions and agrees with the grader at $\kappa = 0.64$ (0.87 excluding one spec-ambiguous task; Section 3). CDS measures divergence of stated beliefs and predicts breakdown, not subtle wrongness; we report that null. The human side of the hazard (whether engineers catch silent failures when an agent reports success) is out of scope and planned as a follow-up study.

Roadmap (not claimed here). The released schema supports, and the next suite version is planned to add: dual expert/non-expert phrasings per task (following Wang et al. (2026), probing the under-specification sensitivity documented by Yuan et al. (2026)); parametric re-instantiation of task families as the contamination control; a multi-part assembly tier with interference checks; ISO 2768 general-tolerance classes per feature under one declared band policy; and the practitioner-coded failure taxonomy with inter-rater agreement. None of these is part of this paper's evidence.

Broader Impact Statement

The direct benefit is a cheap, adoptable harness contract: any CAD copilot can refuse to accept a completion claim until the agent has seen kernel measurements of its final code, and this paper measures what that buys. The benchmark and the released attempt records also give evaluators a domain in which agent overclaiming can be measured without a judge model. The risks run the other way. Absolute rates from 13 canonical parts will be quoted out of context, in both directions: as evidence that agents are unsafe for engineering work, or, because the verified conditions look clean, as evidence that a verification loop makes them safe. Neither follows; the rates are bounded on both sides and specific to one suite, and the human side of the hazard (whether engineers catch what the agent misses) is untested here. A part that passes every check in this suite can still be unmanufacturable or unfit for its function; the checks are model-correctness screens, not design validation, and nothing here should be read as certifying a generated part.

Acknowledgments and Disclosure of Funding

No funding was received for this work. Conflict of interest: Clive Rodrigues provides CAD-for-AI consulting to a company in the mechanical-AI data space. This work uses only open kernels and public data; no proprietary data, tooling, or funding from that engagement was used. Author contributions (CRediT): Carson Rodrigues: conceptualization, software (harness, auto-grader, eval infrastructure), formal analysis, visualization, writing. Clive Rodrigues: conceptualization (engineering requirements), resources (task suite and ground-truth specifications), and investigation (failure-mode taxonomy). Aravind Reddy G: validation (blind expert pass/fail scoring and expert-versus-grader κ), and resources (Siemens NX STEP reproduction of the reference parts).

References

Laksh Advani. From confident closing to silent failure: Characterizing false success in LLM agents, 2026. URL https://arxiv.org/abs/2606.09863.

Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. Generating CAD code with vision-language models for 3D designs, 2024. URL https://arxiv.org/abs/2410.05340.

Mohammadmehdi Ataei, Farzaneh Askari, Kamal Rahimi Malekshan, and Pradeep Kumar Jayaraman. Zero-to-CAD: Agentic synthesis of interpretable CAD programs at million-scale without real data, 2026. URL https://arxiv.org/abs/2604.24479.

Rahul Suresh Babu and Adarsh Agrawal. Self-healing agentic orchestrators for reliable tool-augmented large language model systems, 2026. URL https://arxiv.org/abs/2606.01416.

Akshay Badagabettu, Sai Sravan Yarlagadda, and Amir Barati Farimani. Query2CAD: Generating CAD models using natural language queries, 2024. URL https://arxiv.org/abs/2406.00144.

Jesse Barkley, Rumi Loghmani, and Amir Barati Farimani. CADSmith: Multi-agent CAD generation with programmatic geometric validation, 2026. URL https://arxiv.org/abs/2603.26512.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail?, 2025. URL https://arxiv.org/abs/2503.13657.

Xiaoyu Dong, Zhi Li, and Xiao-Ming Wu. MUSE: Benchmarking manufacturable, functional, and assemblable Text-to-CAD generation, 2026. URL https://arxiv.org/abs/2605.28579.

Yipeng Gao, Lei Shu, Genzhi Ye, Xi Xiong, Ameesh Makadia, Meiqi Guo, Laurent Itti, and Jindong Chen. 3DCodeBench: Benchmarking agentic procedural 3D modeling via code, 2026. URL https://arxiv.org/abs/2606.01057.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. 10.1145/3458723.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks, 2017. URL https://arxiv.org/abs/1705.08500.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing, 2023. URL https://arxiv.org/abs/2305.11738.

Yandong Guan, Xilin Wang, Ximing Xing, Jing Zhang, Dong Xu, and Qian Yu. CAD-Coder: Text-to-CAD generation with chain-of-thought and geometric reward, 2025. URL https://arxiv.org/abs/2505.19713.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks, 2017. URL https://arxiv.org/abs/1706.04599.

Aayush Gupta. ReliabilityBench: Evaluating LLM agent reliability under production-like stress conditions, 2026. URL https://arxiv.org/abs/2601.06112.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet, 2023. URL https://arxiv.org/abs/2310.01798.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues?, 2023. URL https://arxiv.org/abs/2310.06770.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know, 2022. URL https://arxiv.org/abs/2207.05221.

Jean Kaddour, Srijan Patel, Gbètondji Dovonon, Leo Richter, Pasquale Minervini, and Matt J. Kusner. Agentic uncertainty reveals agentic overconfidence, 2026. URL https://arxiv.org/abs/2602.06948.

Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*, 12:1417–1440, 2024. 10.1162/tacl_a_00713.

Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter, 2024. URL https://arxiv.org/abs/2407.01502.

Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2CAD: Generating sequential CAD models from beginner-to-expert level text prompts, 2024. URL https://arxiv.org/abs/2409.17106.

Sangpil Kim, Hyung-gun Chi, Xiao Hu, Qixing Huang, and Karthik Ramani. A large-scale annotated mechanical components benchmark for classification and retrieval tasks with deep neural networks. *Lecture Notes in Computer Science*, pages 175–191, 2020. 10.1007/978-3-030-58523-5_11.

Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. ABC: A big CAD model dataset for geometric deep learning. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9593–9603, 2019. 10.1109/cvpr.2019.00983.

Stephanie Lin, Jacob Hilton, and Owain Evans. Teaching models to express their uncertainty in words, 2022. URL https://arxiv.org/abs/2205.14334.

Dimitrios Mallis, Ahmet Serdar Karadeniz, Sebastian Cavada, Danila Rukhovich, Niki Foteinopoulou, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. CAD-Assistant: Tool-augmented VLLMs as generic CAD task solvers, 2024. URL https://arxiv.org/abs/2412.13810.

Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert, 2020. URL https://arxiv.org/abs/2006.01862.

Felix Ocker, Stefan Menzel, Ahmed Sadik, and Thiago Rios. From idea to CAD: A language model-driven multi-agent system for collaborative design, 2025. URL https://arxiv.org/abs/2503.04417.

Tobias Preintner, Weixuan Yuan, Adrian König, Thomas Bäck, Elena Raponi, and Niki van Stein. EvoCAD: Evolutionary CAD code generation with vision language models, 2025. URL https://arxiv.org/abs/2510.11631.

Carson Rodrigues. Hallucination as context drift: Synchronization protocols for multi-agent LLM systems, 2026. URL https://arxiv.org/abs/2606.21666.

Max Schemmer, Niklas Kuehl, Carina Benz, Andrea Bartos, and Gerhard Satzger. Appropriate reliance on AI advice: Conceptualization and the effect of explanations. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*, pages 410–422, 2023. 10.1145/3581641.3584066.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

Aurel Schüpbach, Raul San Miguel, Julian Ferchow, and Mirko Meboldt. From text to design: a framework to leverage LLM agents for automated CAD generation. *Proceedings of the Design Society*, 5:1893–1902, 2025. 10.1017/pds.2025.10203.

Ari Seff, Yaniv Ovadia, Wenda Zhou, and Ryan P. Adams. SketchGraphs: A large-scale dataset for modeling relational geometry in computer-aided design, 2020. URL https://arxiv.org/abs/2007.08506.

Guijin Son, Jehyun Park, Seyeon Park, Sunghee Ahn, and Youngjae Yu. Self-improving CAD generation agents with finite element analysis as feedback, 2026. URL https://arxiv.org/abs/2605.17448.

Claudio Spiess, David Gros, Kunal Suresh Pai, Michael Pradel, Md Rafiqul Islam Rabin, Amin Alipour, Susmit Jha, Prem Devanbu, and Toufique Ahmed. Calibration and correctness of language models for code. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 540–552, 2025. 10.1109/icse55347.2025.00040.

Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher Manning. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5433–5442, 2023. 10.18653/v1/2023.emnlp-main.330.

Liang Wang, Heng Meng, Zekai Xiang, Jin Liu, Pingyi Zhou, Litao Chen, and Yongqiang Tang. Text2CAD-Bench: A benchmark for LLM-based text-to-parametric CAD generation, 2026. URL https://arxiv.org/abs/2605.18430.

Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: a dataset and environment for programmatic CAD construction from human design sequences. *ACM Transactions on Graphics*, 40(4):1–24, 2021. 10.1145/3450626.3459818.

Rundi Wu, Chang Xiao, and Changxi Zheng. DeepCAD: A deep generative network for computer-aided design models. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6752–6762, 2021. 10.1109/iccv48922.2021.00670.

Haoyang Xie and Feng Ju. Text-to-CadQuery: A new paradigm for CAD generation with scalable large model capabilities, 2025. URL https://arxiv.org/abs/2505.06507.

Yizhe Xie, Congcong Zhu, Xinyue Zhang, Tianqing Zhu, Dayong Ye, Minfeng Qi, Huajie Chen, and Wanlei Zhou. From spark to fire: Modeling and mitigating error cascades in LLM-Based multi-agent collaboration, 2026. URL https://arxiv.org/abs/2603.04474.

Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs, 2023. URL https://arxiv.org/abs/2306.13063.

Jingwei Xu, Chenyu Wang, Zibo Zhao, Wen Liu, Yi Ma, and Shenghua Gao. CAD-MLLM: Unifying multimodality-conditioned CAD generation with MLLM, 2024. URL https://arxiv.org/abs/2411.04954.

Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. An illusion of progress? assessing the current state of web agents, 2025. URL https://arxiv.org/abs/2504.01382.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2022. URL https://arxiv.org/abs/2210.03629.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL https://arxiv.org/abs/2406.12045.

Boxi Yu, Yuxuan Zhu, Pinjia He, and Daniel Kang. UTBoost: Rigorous evaluation of coding agents on SWE-Bench, 2025. URL https://arxiv.org/abs/2506.09289.

Bo Yuan, Zelin Zhao, Petr Molodyk, Bin Hu, and Yongxin Chen. Clarify before you draw: Proactive agents for robust Text-to-CAD generation, 2026. URL https://arxiv.org/abs/2602.03045.

Licheng Zhang, Bach Le, Naveed Akhtar, Siew-Kei Lam, and Tuan Ngo. Large language models for computer-aided design: A survey, 2025. URL https://arxiv.org/abs/2505.08137.

Zheyuan Zhou, Jiayi Han, Liang Du, Naiyu Fang, Lemiao Qiu, and Shuyou Zhang. CAD-Judge: Toward efficient morphological grading and verification for Text-to-CAD generation, 2025. URL https://arxiv.org/abs/2508.04002.

Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala

Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks, 2025. URL `https://arxiv.org/abs/2507.02825`.

Appendix A. Datasheet

Following Gebru et al. (2021); the released repository carries the full version.

Motivation. Created to measure, under an exact geometric oracle, how often LLM agents that write parametric CAD ship a valid, plausible, wrong part while claiming completion, and how well their stated confidence tracks correctness. Built by the authors with no external funding.

Composition. 13 task specifications (5 single-feature, 6 multi-feature, 2 edit-under-constraint) with 122 machine-checkable requirements (58 dimensional `approx`, 13 `range`, 13 `equal`, 13 validity, 22 `cylinder_count` with axial extents, 3 `bolt_circle`); 13 reference solutions; 519 graded agent attempts (four complete-coverage models, plus an incomplete exploratory fifth model that the paper does not use), 156 ablation attempts, and 60 pilot attempts, each a JSON record with task, model, condition, repetition, final code, measured properties, verdict under both oracle versions, failed checks, completion claim, stated confidence, steps, token counts, and per-round context-divergence scores where applicable; multi-turn transcripts for 234 attempts; the 81-row expert scoring sheet. No personal data; no data about people.

Collection. Specifications were written by the authors, one a practising mechanical engineer, and every requirement was mapped to a check before any agent run. Attempts were generated in June 2026 through the vendors' public APIs at provider-default decoding with three repetitions per (task, model, condition) cell; tool-use and multi-agent episodes were capped at four steps and the kernel executor at 60 s per execution. Every attempt was re-executed and regraded under the hardened oracle.

Uses. Evaluating CAD-writing agents for silent failure and claim calibration; testing harness contracts; studying grader leniency. Not for training text-to-CAD models on the reference solutions if the suite is to remain an evaluation.

Distribution and maintenance. Section 7.

Appendix B. Prompts and claim-parsing rules

Verbatim from the released harness (`benchmark/agents.py`); the task specification is appended to each user turn.

System prompt, single-shot and tool-use conditions.

```
You are a senior mechanical CAD engineer. You build parts as parametric
code with the `build123d` Python library. Always assign the final solid
to a top-level variable named `result`. Import everything you use. Work
```

```
in millimetres, and model the part in the conventional orientation:
primary axis / thickness along +Z. Reply with exactly one ```python code
block. When you are confident the part satisfies the specification,
also write - after and outside the code block, on their own lines -
`DONE` and `CONFIDENCE: <0..1>`, your calibrated probability that every
requirement is met.
```

Tool-use user preamble (prepended to the specification).

```
Declare DONE only after reviewing the measurements of your latest code.
```

Tool-use feedback template (one message per executed revision).

```
Kernel-measured properties of your solid:
<JSON with valid_solid, bbox, volume, com, solid_count, cylinders>
If this meets the spec, reply DONE + CONFIDENCE. Otherwise send a
corrected ```python block.
```

Message when DONE accompanies unmeasured code (measure-then-claim contract).

```
Your DONE was not accepted yet: review the measurements above, then
reply DONE + CONFIDENCE (no code) if the part meets the spec, or send a
corrected ```python block.
```

Planner system prompt.

```
You are the PLANNER in a mechanical CAD team. From the specification,
produce a short numbered build plan (features, order of operations,
datums) for a `build123d` Python implementation. Work in millimetres.
Do NOT write code. End your reply with one line `DIMS: {"name": value,
...}` - a flat JSON object naming every governing dimension in mm
(numeric values only).
```

Coder system prompt.

```
You are the CODER in a mechanical CAD team. Implement the plan with the
`build123d` Python library. Always assign the final solid to a
top-level variable named `result`. Import everything you use. Work in
millimetres. Reply with exactly one ```python code block, then one line
`DIMS: {"name": value, ...}` - a flat JSON object of the governing
dimensions (mm) your code implements.
```

Checker system prompt.

```
You are the CHECKER in a mechanical CAD team. You receive the original
specification and kernel-measured properties of the part the coder
built. Judge whether the part satisfies every requirement. Reply with:
`VERDICT: PASS` or `VERDICT: REVISE`, a one-line reason (if REVISE, say
exactly what to change), one line `DIMS: {"name": value, ...}` - the
governing dimensions (mm) you believe the measured part actually has -
and one line `CONFIDENCE: <0..1>`, your calibrated probability that
every requirement is met.
```

Parsing rules. A completion claim counts only when the token `DONE` starts its own line outside any code block (so "this is NOT DONE" never counts); under the measure-then-claim contract a `DONE` that accompanies fresh, unmeasured code is rejected with the message above and the episode continues. Confidence is read from a `CONFIDENCE: <0..1>` line; a missing or unparsable value is recorded as no stated confidence and excluded from calibration statistics. The checker's verdict is read from `VERDICT: PASS|REVISE`, and each role's stated dimensions from a `DIMS: {...}` line of flat JSON in millimetres, which is what the Context Divergence Score compares. Tool-use and multi-agent episodes are capped at four steps; the kernel executor is capped at 60 s per execution.

Appendix C. Reproducibility checklist

- **Code and data:** all released (Section 7); one script regrades every stored attempt offline and one script produces every statistic, table, and figure.
- **Models:** API identifiers exactly as called: `claude-haiku-4-5` and `claude-sonnet-4-6` (Anthropic API; the sonnet ablation arm as `anthropic/claude-sonnet-4-6` via OpenRouter), `deepseek/deepseek-v3.2` and `qwen/qwen3.7-max` (OpenRouter), all called in June 2026 except the sonnet and qwen ablation arms (September 2026) (an incomplete gpt-5.5 run ships in the release, unused here); provider-default decoding, 4,096 output tokens, three repetitions per cell, no seed control.
- **Harness:** prompts and parsing rules verbatim in Appendix B; tool-use and multi-agent capped at four steps; executor timeout 60 s; measure-then-claim contract as defined in Section 3; the legacy contract is retained as the ablation arm.
- **Statistics:** Wilson intervals on rates; task-cluster bootstrap (4,000 resamples) on contrasts; exact McNemar over (task, model, repetition) triples; task-clustered GEE; Fisher exact for the 156-vs-156 ablation; all computed by the released script and reproduced from the raw records before submission.
- **Grader validation:** calibration gate on reference solutions; blind expert scoring of 81 parts with the sheet and the κ computation released; oracle-hardening audit with verdict flips reported in both directions.
- **Pre-registration:** the study plan with predictions P1–P3 is in the repository; Section 8 reports which held.
- **Compute and cost:** the whole corpus is API calls at a few thousand tokens per attempt; no GPU. Regrading and the tolerance sweep run on a laptop in under an hour.